

RoK4 Base-Velocity Estimator 구조 문서

작성일: 2026-08-18
최종 업데이트: 2026-08-19
대상 브랜치: yunho/concurrent-state-estimator
대상 저장소: RoK4 repository root ($\text{\${R0K4_LAB_ROOT}}$)
기준 환경: Isaac Lab v2.3.2, Isaac Sim 5.1.0, env_isaacrlab

문서 목적

이 문서는 RoK4 policy와 body-frame base linear velocity estimator를 동시에 학습하는 구조를 설명한다. 목표는 실기에서 직접 측정하기 어려운 $\text{base_lin_vel_b}=[v_x, v_y, v_z]$ 를 proprioceptive history로 추정해 Actor가 급정지와 외란 이후의 실제 움직임을 command와 구분할 수 있게 만드는 것이다.

이번 첫 실험에서는 estimator 자체의 효과를 분리하기 위해 reward, command population, periodic freeze, push DR, stable-standing gate를 추가로 바꾸지 않는다. 기존 yunho/privileged-observation 상태에서 새 브랜치를 만들었으며 이전 checkpoint를 resume하지 않고 fresh training한다.

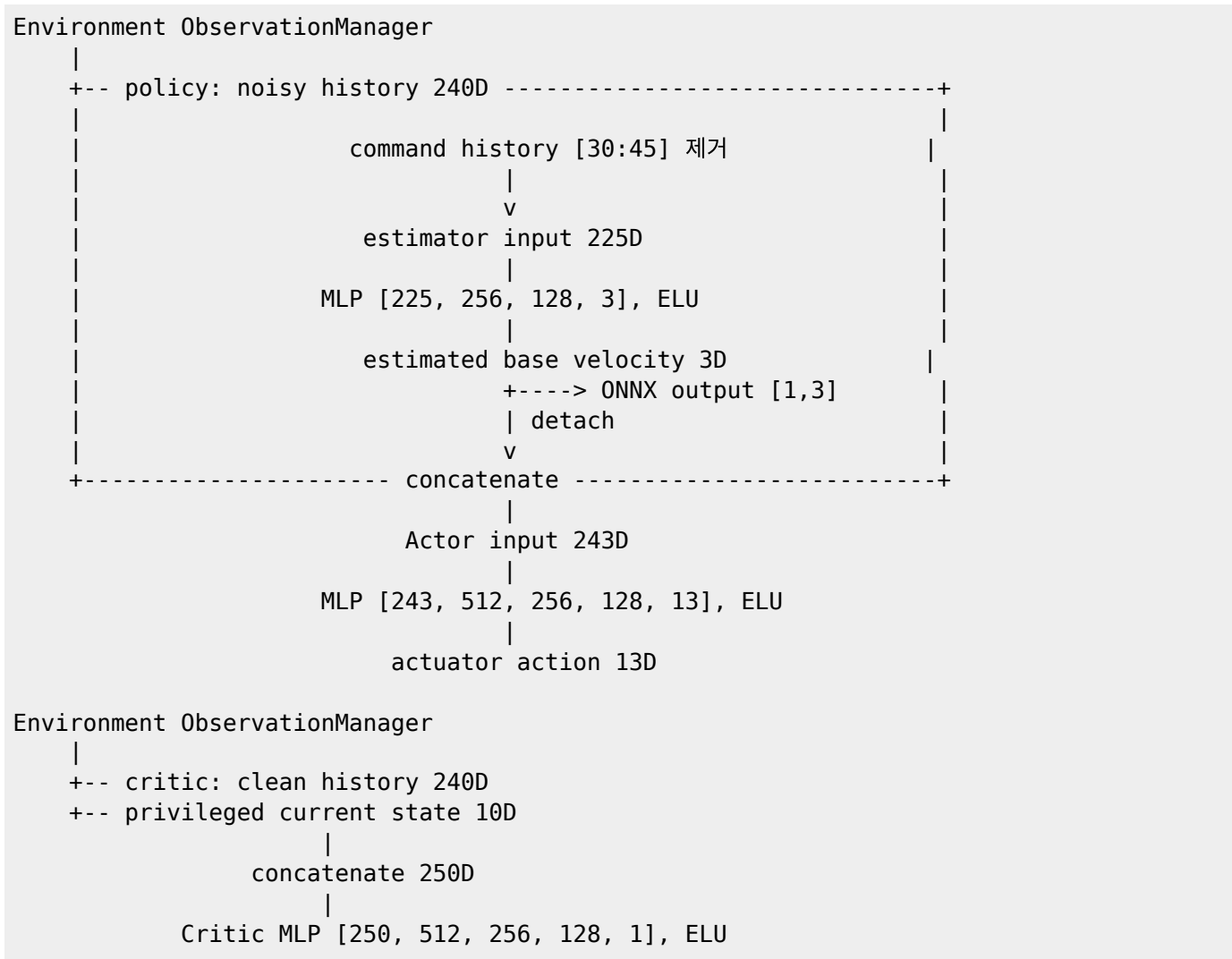
핵심 계약

외부 observation dimension과 action dimension은 바뀌지 않는다. ONNX는 하나의 16D tensor가 아니라 의미와 단위가 분리된 두 개의 named output을 제공한다.

deployment input	obs [1,240]
control output	actions [1,13]
estimator diagnostic output	estimated_base_lin_vel_b [1,3]
Critic input	250D clean/privileged training state

Estimator와 Actor는 하나의 exported policy 내부에 포함된다. ONNX 소비자는 여전히 [1, 240] 만 입력하며 별도의 velocity-estimator node나 243D 입력을 만들지 않는다. 대신 Actor가 실제로 사용한 동일한 추정값을 두 번째 출력으로 읽어 Sim2Sim/Sim2Real에서 기록할 수 있다.

전체 network 구조



Actor observation 240D

Isaac Lab ObservationManager는 각 term의 5-step history를 먼저 flatten한 뒤 term들을 concatenate한다.

Actor policy layout

index	term	shape	corruption
0:15	base_ang_vel	5 x 3	uniform noise [-0.2, 0.2]
15:30	projected_gravity	5 x 3	uniform noise [-0.05, 0.05]
30:45	velocity_commands	5 x 3	없음
45:110	actuator_pos	5 x 13	uniform noise [-0.01, 0.01] rad
110:175	actuator_vel	5 x 13	uniform noise [-1.5, 1.5] rad/s
175:240	last_action	5 x 13	없음

Actor normalizer는 이 240D를 원소별로 normalize한다. Estimator도 실기 Actor가 받는 것과 동일한 noisy/normalized history를 사용한다. Clean Critic history를 estimator에 넣지 않는다.

Estimator input 225D와 command 제외

Estimator 입력은 normalized 240D에서 command history 30:45 를 제외한 225D다.

$$o_t^{est} = [\bar{o}_t, 0 : 30, \bar{o}_t, 45 : 240] \in \mathbb{R}^{225}$$

여기서 bar 는 empirical normalization 이후 값을 뜻한다. Command를 제외하는 이유는 estimator가 실제 state 를 추정하지 않고 desired command를 복사하는 shortcut을 막기 위해서다. Actor는 원래 240D 안에서 command를 계속 받으므로 제어 목표는 사라지지 않는다.

이 선택은 TRON1의 구현처럼 estimator와 Actor의 command 경로를 분리하는 방향이다. 논문의 한 구현처럼 current command를 estimator에 넣는 것도 가능하지만, RoK4의 첫 목적은 command=0 인 급정지/외란 상황에서 실제 base motion을 구별하는 것이므로 command-free estimator를 사용한다.

Estimator target과 loss

Supervised target은 current privileged observation의 첫 세 값이다.

$$y_t = v_{base, b}^{sim} = [v_x, v_y, v_z] \in \mathbb{R}^3$$

Estimator는 MLP 225 -> 256 -> 128 -> 3 이고 hidden activation은 ELU, output은 linear다.

$$\hat{v}_t = f_{\phi}(o_t^{est})$$

학습 loss는 batch와 세 축 전체의 MSE다.

$$L_{est} = \frac{1}{3B} \sum_{i=1}^B \|\hat{v}_i - v_i^{sim}\|_2^2$$

Estimator optimizer는 PPO optimizer와 분리된 Adam이고 learning rate는 1e-3, gradient norm cap은 1.0 이다. PPO의 adaptive learning rate가 변해도 estimator learning rate는 독립적으로 유지된다.

TensorBoard에는 학습 목적함수 Loss/estimator_mse 와 물리 단위 평가값을 함께 기록한다.

Metrics/estimator/rmse_vx	[m/s]
Metrics/estimator/rmse_vy	[m/s]
Metrics/estimator/rmse_vz	[m/s]
Metrics/estimator/rmse_total	[m/s]

축별 RMSE와 total RMSE는 다음과 같다.

$$RMSE_j = \sqrt{\frac{1}{B} \sum_i (\hat{v}_{i,j} - v_{i,j})^2}$$

$$RMSE_{total} = \sqrt{MSE_x + MSE_y + MSE_z}$$

MSE는 제곱근이 없어 최적화에 사용하고, RMSE는 원래 단위 m/s 로 읽기 위해 logging에 사용한다.

PPO와 estimator gradient 분리

Actor input은 normalized policy history와 estimated velocity를 concatenate한 243D다.

$$a_t = \pi_{\theta}([\bar{o}_{policy}^t, stopgrad(\hat{v}_t)])$$

stopgrad 는 PyTorch의 detach() 다. 이 때문에 PPO surrogate/value/symmetry 경로는 estimator parameter phi 를 바꾸지 않는다. Estimator는 오직 supervised MSE로만 갱신된다.

```
PPO optimizer
updates: action MLP, Critic MLP, action std
excludes: estimator MLP

Estimator optimizer
updates: estimator MLP only
objective: base velocity MSE
```

각 PPO mini-batch에서 먼저 PPO update를 수행하고, 같은 observation batch로 estimator MSE update를 한 번 수행한다. 따라서 rollout을 별도로 다시 수집하지 않으며, policy와 estimator는 같은 학습 iteration의 state distribution을 본다.

Critic 250D는 그대로 유지

Critic은 기존 asymmetric 구조를 그대로 사용한다.

```
clean proprioceptive history 240D
+ current privileged state    10D
-----
Critic input                  250D
```

Privileged 10D layout은 다음과 같다.

```
[0:3]   true base_lin_vel_b [vx, vy, vz]
[3:4]   base height
[4:6]   left/right Foot_Link height
[6:8]   left/right contact flag
[8:10]  left/right current air time
```

Critic에는 true velocity가 있으므로 value estimation은 privileged state를 계속 활용한다. 이번 v1에서는 estimated velocity를 Critic에 다시 붙이지 않는다. Estimator 성능과 Actor 효과를 먼저 분리하기 위한 선택이다.

Symmetry augmentation

기존 좌우 symmetry augmentation은 estimator 학습에도 그대로 적용된다. Mirror observation은 동일한 estimator를 통과하고 privileged target의 lateral component를 반전한다.

$$[v_x, v_y, v_z] \rightarrow [v_x, -v_y, v_z] \text{ (left-right mirror)}$$

원본과 mirror sample을 같은 estimator MSE batch에서 학습한다. Network를 두 개 만들지 않으며 left/right estimator weight도 공유한다. Command history는 원본과 mirror 모두 estimator 입력에서 제거된다.

Checkpoint와 resume

Checkpoint에는 다음 상태가 저장된다.

```
model_state_dict
  actor policy MLP
  estimator MLP
  critic MLP
  actor/critic empirical normalizers
  action standard deviation

optimizer_state_dict          PPO optimizer
estimator_optimizer_state_dict estimator Adam
iter
infos
```

Estimator 없는 이전 checkpoint는 Actor 첫 층 구조도 240 -> ... 로 달라 새 branch에서 그대로 resume할 수 없다. 이번 실험은 fresh training을 사용한다. 새 checkpoint끼리 resume할 때는 두 optimizer 상태도 함께 복원된다.

Play, Teleop, JIT, ONNX

play.py 와 play_teleop.py 도 RoK40nPolicyRunner 를 사용한다. 표준 exporter는 Actor의 첫 linear layer를 외부 input dimension으로 해석하므로 내부 243D Actor에 그대로 사용할 수 없다. RoK4 exporter는 normalizer, 225D estimator, detach, 243D action MLP를 하나의 graph로 묶고 외부 dummy input을 명시적으로 240D로 둔다.

```
external obs [1,240]
-> empirical normalizer
-> command-free estimator [1,225] -> [1,3]
   +-> estimated_base_lin_vel_b [1,3]
   |
   +-> detach -> concatenate [1,243]
       -> action MLP
       -> actions [1,13]
```

따라서 기존 Sim2Sim/Sim2Real node의 ONNX input shape를 바꾸지 않는다. ONNX output은 다음 두 tensor 다.

policy.onnx interface

이름	shape	단위	용도
actions	[1,13]	normalized raw action	actuator command 생성에 사용
estimated_base_lin_vel_b	[1,3]	m/s	Actor가 믿은 body-frame $[v_x, v_y, v_z]$ 기록

두 output은 [1,16] 으로 합쳐지지 않는다. 제어 코드는 actions 전체 13개를 그대로 사용하므로 [:13] 같은 slicing도 필요 없다. 두 번째 output은 첫 번째 action을 계산할 때 사용한 바로 그 estimator tensor이며, 진단을 위해 estimator를 다시 실행한 별도 결과가 아니다. Output을 하나 더 공개해도 action 계산 graph나 값은 바뀌지 않는다.

Python ONNX Runtime 사용 예시는 다음과 같다.

```
actions, estimated_velocity = session.run(
    ["actions", "estimated_base_lin_vel_b"],
    {"obs": observation_240},
)

# actions.shape == (1, 13): actuator command에 사용
# estimated_velocity.shape == (1, 3): logging/plot에 사용
```

Sim2Sim은 추정값과 simulator ground truth base_lin_vel_b 를 함께 기록한다.

$$e_t = \hat{v}_{base, b, t} - v_{base, b, t}^{sim}$$

$$RMSE_j = \sqrt{\frac{1}{N} \sum_{t=1}^N (\hat{v}_{t,j} - v_{t,j}^{sim})^2}$$

v_x, v_y, v_z 축별 RMSE와 total RMSE를 계산하면 TensorBoard의 estimator metric과 같은 물리 단위로 Sim2Sim 성능을 비교할 수 있다. Sim2Real에는 simulator ground truth가 없으므로 추정값을 ROS topic, CSV 또는 plot으로 기록한다. Motion capture, VIO 또는 별도 reference estimator가 있으면 timestamp를 맞춰 같은 방식으로 비교한다.

TorchScript policy.pt 는 기존 소비자와의 호환성을 위해 action-only [1,13] interface를 유지한다. 두 출력이 필요한 Sim2Sim/Sim2Real 배포에는 policy.onnx 를 사용한다. 기존 estimator checkpoint는 재학습하지 않고 새 exporter로 다시 export할 수 있다. 다만 ONNX output이 하나라고 가정한 기존 runtime은 두 output 이름을 읽도록 한 번 수정해야 한다.

Teleop estimator 화살표

play_teleop.py 는 policy step마다 현재 240D observation으로 Actor와 동일한 estimator를 실행하고, 그 결과의 평면 성분 $[\hat{v}_x, \hat{v}_y]$ 을 주황색 화살표로 표시한다. 기존 command debug visualization과 좌표계 및 길이 배율을 맞췄으므로 방향과 길이를 직접 비교할 수 있다.

Teleop velocity arrows

색상	값	좌표계	의미
녹색	command $[v_x, v_y]$	body frame	사용자가 요구한 평면 선 속도
파란색	simulator root_lin_vel_b[0:2]	body frame	Isaac Sim ground truth 실제 속도
주황색	estimator $[\hat{v}_x, \hat{v}_y]$	body frame	같은 policy observation에서 Actor가 사용한 추정 속도

세 화살표는 body-frame XY 방향을 robot orientation으로 world에 회전시켜 로봇 위에 그린다. 길이는 모두 $3 * \sqrt{v_x^2 + v_y^2}$ 에 같은 prototype scale을 적용한다. 주황색 화살표만 기존 두 화살표보다 0.08 m 높게 배치해 값이 비슷할 때도 색상을 확인할 수 있게 한다. Command에는 v_z 목표가 없으므로 공정한 비교를 위해 화살표는 XY만 표현한다. Estimator가 함께 출력하는 $\hat{v}_z$ 는 ONNX diagnostic output 또는 TensorBoard RMSE로 확인한다.

이 화살표는 진단용이며 observation, action, reward, estimator loss, ONNX graph를 변경하지 않는다. Teleop에서 estimator를 시각화하기 위해 한 번 더 forward하지만 환경이 하나이므로 제어 주기 영향은 무시할 수 있는 수준이다. 표시된 값은 action 계산과 동일한 current observation 및 같은 estimator weight를 사용한 결정적 출력이다.

관련 파일

Estimator implementation files

파일	역할
scripts/rsl_rl/rok4_ppo.py	estimator Actor, Actor-Critic, 별도 MSE optimizer, RMSE logging, checkpoint, fused exporter
config/rok4/agents/rsl_rl_ppo_cfg.py	estimator hidden layer, learning rate, command-history slice와 custom class 이름 설정
config/rok4/flat_env_cfg.py	noisy 240D policy, clean 240D critic, current privileged 10D 제공
mdp/symmetry.py	policy history와 privileged velocity target의 좌우 mirror
scripts/rsl_rl/_run_isaacclab_rsl.py	Train/Play에 custom runner를 주입하고 Play export를 fused exporter로 교체하며 Teleop 추정 속도를 visualizer에 전달
scripts/rsl_rl/_velocity_estimate_visualizer.py	Teleop estimator body-frame XY를 주황색 world marker로 표시
tests/test_state_estimator.py	dimension, command 제외, gradient 분리, PPO update, TorchScript와 ONNX 두 출력 및 action 동일성 검증
tests/test_rsl_rl_wrapper.py	Train/Play runner 및 exporter source patch 회귀 검증

첫 학습과 확인 순서

```
export ROK4_LAB_ROOT="${HOME}/rok4_lab"
export ISAACLAB_ROOT="${HOME}/IsaacLab"
cd "${ISAACLAB_ROOT}"
conda activate env_isaactlab

./isaactlab.sh -p ${ROK4_LAB_ROOT}/scripts/rs_l_rl/train.py \
  --task RoK4-Isaac-Velocity-Flat-v0 \
  --num_envs 4096 \
  --max_iterations 20000 \
  --headless \
  --run_name concurrent_vel_estimator_fresh20k
```

확인 우선순위는 다음과 같다.

1. Loss/estimator_mse가 안정적으로 감소하는가
2. Metrics/estimator/rmse_vx, vy, vz가 감소하는가
3. velocity tracking과 symmetry reward가 기존 baseline보다 무너지지 않는가
4. Play/Teleop exported policy.onnx input이 obs [1,240] 인가
5. ONNX output이 actions [1,13] 와 estimated_base_lin_vel_b [1,3] 로 분리되는가
6. ONNX action이 동일 checkpoint의 action-only exporter와 수치적으로 같은가
7. Sim2Sim ground truth와 estimator output의 축별/total RMSE가 허용 범위인가
8. 급정지와 push에서 Actor가 estimated velocity를 활용하는가

후속 단계

Estimator가 충분히 정확해진 뒤에만 다음 실험을 별도로 진행한다.

```
command == 0 and estimated motion small -> stable standing gate on
command == 0 and estimated motion large -> recovery, standing pose constraint off
walking command and external disturbance -> recovery step allowed
```

Stable gate threshold, hysteresis, dwell time은 estimator RMSE와 실기 delay/noise를 본 뒤 정한다. 이번 branch에서는 미리 임계값을 넣지 않는다. 그래야 estimator 자체의 학습 품질과 locomotion 효과를 먼저 판단할 수 있다.